宁波大学

本科实验报告

姓名：	梁航锋
学院：	信息科学与工程学院
系：	计算机科学与技术系
专业：	计算机科学与技术
学号：	216001912
指导教师：	董理

2024 年 3 月 14 日

宁波大学实验报告

课程名称：机器学习 实验类型：验证性

实验项目名称：实验一：模型泛化性理解

学生姓名：梁航锋 专业：计算机科学与技术 学号：216001912

同组学生姓名：

指导老师：董理

实验地点：计算机组成实验室 201 实验日期：2024 年 3 月 13 日

一、实验目的和要求

1. 学习掌握相关理论知识，例如模型的泛化性定义、过拟合、欠拟合等，并通过编程，从实践的角度提高对模型泛化性的理解；

2. 掌握 anaconda 环境、conda 工具的安装与使用，并学习使用 sklearn、matplotlib 等常见 Python 库函数；

3. 掌握机器学习模型训练的常规流程，包括数据集处理与划分、模型的训练、模型的验证等步骤，并理解学习各步骤的代码实现方法；

4. 总结归纳实验流程，理解过拟合、欠拟合等现象的发生原理、实验状况以及解决方案。

二、实验内容和原理

1. 使用 sklearn 框架实现回归任务中的拟合过程，复现模型欠拟合、过拟合、正好拟合的状态，并适当进行分析。

2. 基于 Python 及其库函数完成实验，将实验中产生的相关可视化图像、实验源码（包含相关语句注释）进行记录。

3. 完成实验报告的攥写，并及时提交实验报告。

三、实验环境

操作系统：

Windows 11 Version 22H2

集成开发环境：

1. Anaconda3 with Jupyter Notebook
2. PyCharm 2022.2.2 (Professional Edition)

相关 Python 库：

1. python3.9
2. numpy
3. matplotlib（3.8.3）
4. sklearn（scikit-learn1.4.1）
5. pandas（2.2.1）

四、操作方法与实验步骤

1. 环境搭建

本次实验采用 conda 进行虚拟环境的搭建，使用以下命令进行虚拟环境的创建：conda create -n envname python=3.x，其中 envname 为环境名称，为方便标识以及后续实验的进行，将虚拟环境取名为 nbuml，如图 4.1 所示。

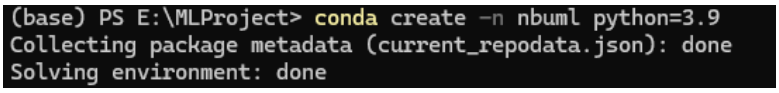


图 4.1 conda 虚拟环境创建

使用 conda env list 命令查看计算机中所创建的虚拟环境，可以查看到新创建的虚拟环境 nbuml，如图 4.2 所示。

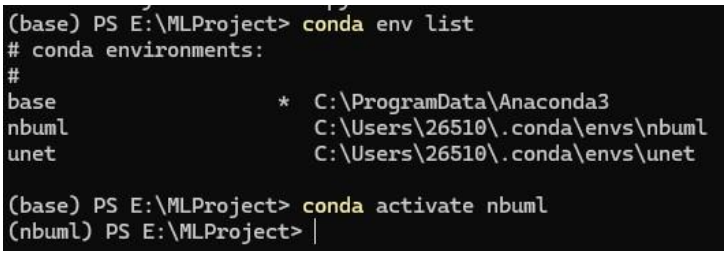


图 4.2 conda 虚拟环境查看

使用 pip 工具进行实验所需的相关 Python 库的安装，安装命令与过程如图 4.3 所示。

2. 集成开发环境的使用

为方便后续实验代码的编写与维护，本次实验使用 PyCharm 作为集成开发工具，并使用配套的 Jupyter Notebook 进行代码的调试与运行。

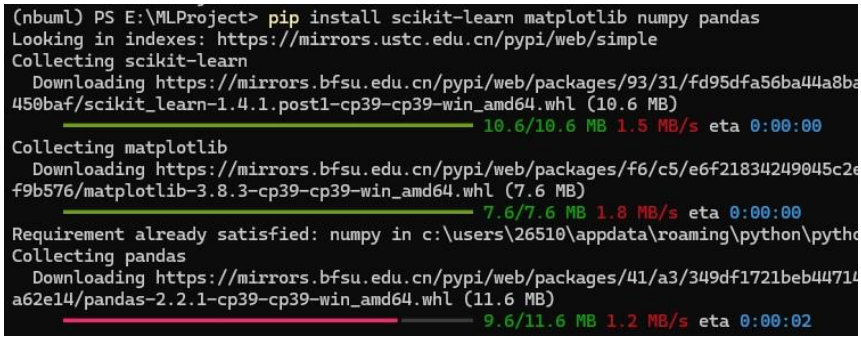


图 4.3 使用 pip 工具安装所需库

首先进行项目的新建，并设置实验环境为原先配置好的虚拟环境，使用其中的解释器进行 Python 代码的解析与运行，相关配置如图 4.4 所示。



图 4.4 使用 PyCharm 进行项目创建

3. 实验内容

（1）数据生成

生成随着特定函数分布的几百个随机数据点，包括训练数据与测试数据，例如 y=sin(x)。具体方法为生成[-3,3]区间的随机数据 100 个，作为样本特征，并将其作 sin 正弦运算，加上一个随机噪声，以作为样本标签。

（2）数据可视化

使用 matplotlib 库进行图像的绘制，具体代码见下文。首先对训练集、测试集的数据进行可视化图像的绘制。

（3）数据维度改变

由于原本数据集的样本特征维数只有 1，即数据的形状为[n,1]，其中 n 为样本数，同时考虑到本次实验主要探究不同模型的泛化性差异，因此考虑将特

征映射到高维的多项式特征空间中，使用 PolynomialFeature 进行多项式特征的生成，实例化对象时传入多项式的维数，随后调用对象的 fit_transform 即可完成数据的映射。实验将分别对度为 2、4、25 的多项式特征模型的拟合程度进行测试。

（4）数据拟合

使用 LinearRegression 进行线性回归模型的实例化，调用 fit 函数，传入训练集特征与标签，调用 predict 函数，基于训练好的线性回归模型来预测。

（5）模型测试

将测试集的数据完成特征映射后，调用 predict 函数将模型应用于测试集上进行验证，并进行训练结果的评估。

（6）可视化拟合结果

将（3）步骤中所提及的度为 2、4、25 的多项式特征模型均在训练集上进行拟合，并在测试集上验证，并绘制图像，分析拟合结果。

五、实验数据记录和处理

该部分将按照代码逻辑与以上实验步骤进行代码的划分，并记录。

```
# 相关库导入
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
```

```
# 生成测试集与训练集
rnd  = np.random.RandomState(42)
# 训练集，样本数100
x_train = rnd.uniform(-3, 3, size=100)
y_train = np.sin(x_train) + rnd.normal(size=len(x_train)) / 3
# 测试集，样本数100
x_test = rnd.uniform(-3, 3, size=100)
y_test = np.sin(x_test) + rnd.normal(size=len(x_test)) / 3
```

```
# 训练集、测试集数据绘制
fig, ax1 = plt.subplots(1)
# 训练集数据，图像为绿色，标签为train dataset
ax1.plot(x_train, y_train, 'o', c='g', label='train dataset')
# 测试集数据，图像为蓝色，标签为test dataset
```

```
ax1.plot(x_test, y_test, 'o', c='b', label='test dataset')
# 将标签放置于最佳位置
ax1.legend(loc='best')
# 设置其余图像选项，包括横纵坐标标签、图像标题
ax1.set_ylabel('y')
ax1.set_xlabel('x')
ax1.set_title("Train Dataset Display")
# 图像尺寸调整
plt.tight_layout()
plt.show()
```

```
# 数据维度改变：变为 shape 为[n,f]，其中 n 为样本数，f 为特征数，本例为 1
x_train_reshape = x_train.reshape(-1, 1)
x_test_reshape = x_test.reshape(-1, 1)
# 将数据集进行特征转换，调整为多项式特征，其中度为 4 的多项式具有 5 个特征值，
故 shape 为[n,5]
poly = PolynomialFeatures(degree=4)
x_train_poly = poly.fit_transform(x_train_reshape)
```

```
# 模型实例化：线性回归模型
lr = LinearRegression()
# 在训练集上训练数据
lr.fit(x_train_poly, y_train)
y_predict = lr.predict(x_train_poly)
# 在测试集上测试模型
x_test_poly = poly.fit_transform(x_test_reshape)
y_test_predict = lr.predict(x_test_poly)
```

```
# 绘制图像：训练集的拟合情况
fig, ax1 = plt.subplots(1)
ax1.plot(x_train, y_train, 'o', c='k',label='train point dataset')
ax1.plot(np.sort(x_train), y_predict[np.argsort(x_train)], linewidth=
2, c='g', label='train regression')
ax1.legend(loc='best')
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_title('Train Result Display')
plt.tight_layout()
plt.show()
plt.close()
```

```
# 绘制图像，查看测试集的验证情况
fig, ax1 = plt.subplots(1)
ax1.plot(x_test, y_test, 'o', c='r', label='test point dataset')
ax1.plot(np.sort(x_test), y_test_predict[np.argsort(x_test)], linewid
th=2, c='b', label='test regression')
ax1.legend(loc='best')
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.set_title('Test Result Display')
plt.tight_layout()
plt.show()
plt.close()
```

六、实验结果与分析

该部分将对实验代码产出的相关图像进行记录与分析。

1. 数据的分布

实验的第一步进行了数据的随机生成，包括训练集与测试集，这里通过随机生成的方法确保了训练集与测试集的互斥。数据分布图如图 6.1 所示。

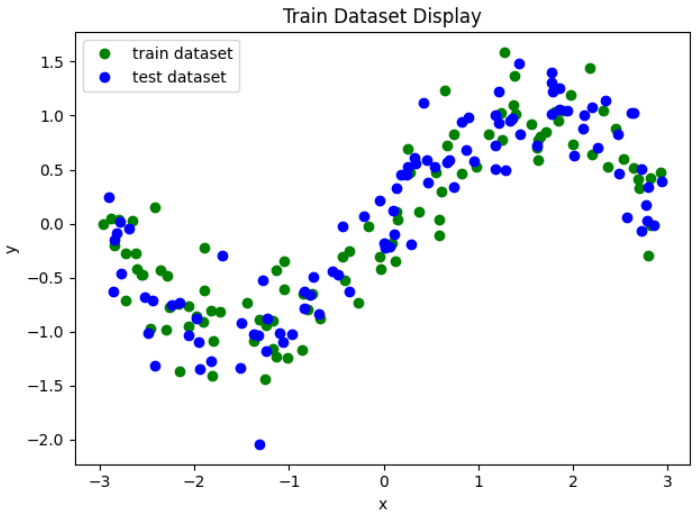


图 6.1 实验数据的分布

2. 正常拟合的模型（度为 4 的多项式特征）

对于度为 4 的多项式特征，其特征数目为 5，样本特征的形状为[100, 5]，通过训练集的训练以及测试集的验证，其效果如图 6.2 所示。

观察可视化结果可以发现，模型拟合的曲线接近正弦函数，与数据生成时的操作（正弦运算后加噪声）相匹配，模型拟合正常。

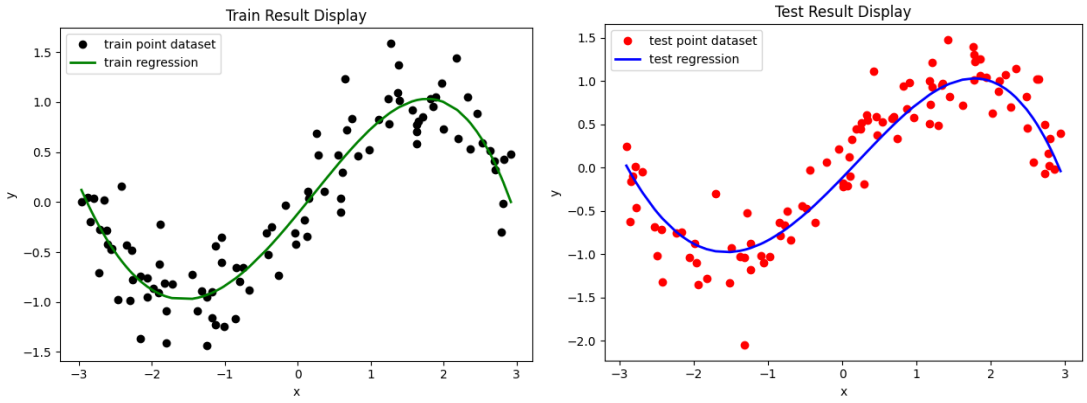


图 6.2 模型拟合与验证（多项式度数=4）

3. 欠拟合的模型（度为 2 的多项式特征）

对于度为 2 的多项式特征，其特征数目为 3，样本特征的形状为[100,3]，通过训练集的训练以及测试集的验证，其效果如图 6.3 所示。

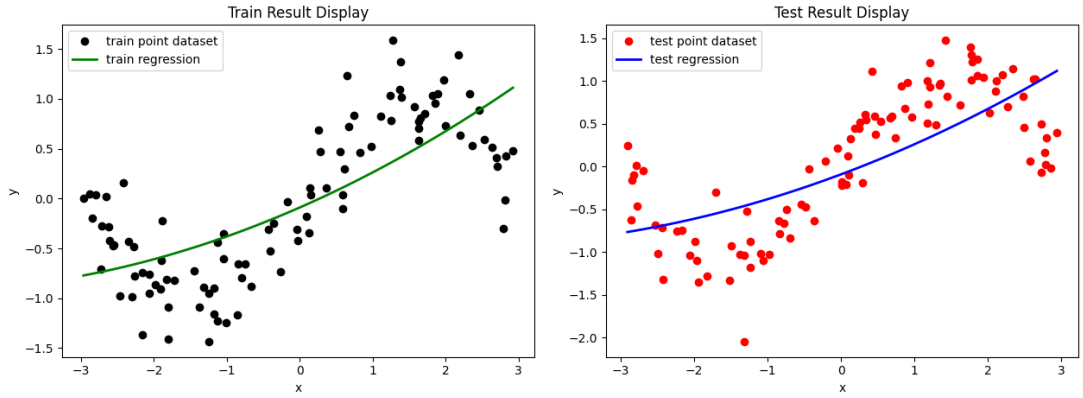


图 6.3 模型拟合与验证（多项式度数=2）

观察拟合结果可以发现，拟合曲线过于简单，并不能很好地拟合给定的数据，对于度为 2 的多项式，即二次函数形式，无法精确地描述基于正弦函数生成的数据，这便产生了欠拟合的现象。

4. 过拟合的模型（度为 25 的多项式特征）

对于度为 25 的多项式特征，其特征数目为 26，样本形状为[100,26]，通过训练集的训练以及测试集的验证，其效果如图 6.4 所示。

观察拟合结果可以发现，拟合曲线虽然能够较好地接近绝大部分数据，产生较小的训练误差，但可以看出模型过于复杂，而实际上在产生数据时使用的为正弦函数，与拟合结果差距较大。多次运行实验代码可以发现，模型的拟合受到噪声等随机因素的干扰极大，这便是模型训练的过拟合现象。根据“奥卡姆剃刀”原则，这类复杂的模型通常不被选择。

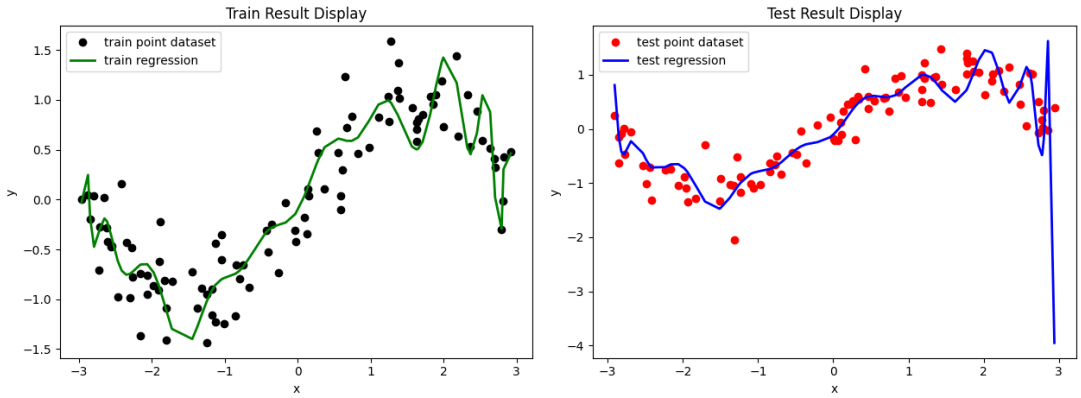


图 6.4 模型拟合与验证（多项式度数=25）

七、讨论、心得

本次实验基于 Python 与 sklearn 库进行了基本的机器学习实验，包括随机数据的生成与划分、多项式特征的映射、特征形状的调整、数据的可视化绘图、线性回归模型的拟合等常见内容，同时也掌握了基于 conda 的虚拟环境配置方法，以便于多个独立的项目环境之间的不相互干扰。在此基础上，通过调整多项式特征的维数来产生不同的模型，以展现模型的正常拟合、过拟合、欠拟合等现象，从而更好地学习掌握其中的原理与差异。

对于一些结构较为简单的模型，例如实验过程中配置的度为 2 的多项式特征，由于特征维数较小，模型结构简单，所能够表征的信息层次较浅，模型无法学习到数据中的相关特征，从而出现了欠拟合的现象；倘若模型结构较为复杂，例如实验中度为 25 的多项式特征，维数较高，能够拟合出极为复杂的数据表示，在此基础上若充分学习，模型则会尽可能的贴近每一个数据样本，这便产生了过拟合的现象。过拟合与欠拟合均为无法很好地从数据中得到学习的体现，因此在模型的训练过程中，要尽可能避免欠拟合与过拟合现象，这可以通过“奥卡姆剃刀”原则、提高数据集质量等方法实现。